PharmaShop

Digital Twin — Centre d'Assistance Client

Rapport Technique

Base de données PostgreSQL & Dashboard BI

Auteur: Samar

Date: 09 November 2026

Projet: Système multi-agents pour centre d'assistance

Domaine: E-commerce & Parapharmacie (Tunisie)

Technologies: PostgreSQL, Python, Streamlit, React, FastAPI

■ Table des matières

• 1. Introduction et contexte du projet
• 2. Architecture de la base de données PostgreSQL
• 2.1. Tables principales
• 2.2. Vues Business Intelligence
• 3. API Dashboard BI
• 3.1. Endpoints FastAPI
• 3.2. Intégration frontend-backend
• 4. Dashboard Streamlit
• 4.1. Architecture et fonctionnalités
• 4.2. Pages et visualisations
• 5. Dashboard React intégré
• 6. Diagramme d'architecture
• 7. Métriques et KPIs disponibles
• 8. Conclusion et perspectives

1. Introduction et contexte du projet

Ce rapport documente la contribution apportée au projet **Digital Twin** du centre d'assistance client pour la parapharmacie en ligne tunisienne **MaPara**. Le projet implémente un système multi-agents orchestrant six composants principaux : Client Simulator, Customer Advisor (RAG), Quality Analyst, Workforce Manager, Result & Optimization et Digital Twin.

Ma contribution se concentre sur deux piliers essentiels :

- **Base de données PostgreSQL** : Conception et implémentation du schéma complet incluant 6 tables relationnelles et 7 vues optimisées pour l'analyse BI
- **Dashboards BI** : Développement de deux interfaces de visualisation (Streamlit standalone et React intégré) avec endpoints API FastAPI

Objectifs du système de persistence et de visualisation :

- Traçabilité complète des interactions client-conseiller simulées
- Analyse en temps réel des performances du centre d'assistance
- Identification des points faibles et opportunités d'amélioration
- Support à la prise de décision opérationnelle basée sur les données
- Visualisation intuitive des KPIs pour différents profils d'utilisateurs

2. Architecture de la base de données PostgreSQL

La base de données **callverse** constitue le cœur du système de persistence. Elle stocke l'intégralité du cycle de vie des conversations : depuis la génération du profil client jusqu'aux décisions du Workforce Manager, en passant par les évaluations qualité. L'architecture relationnelle garantit l'intégrité référentielle et optimise les requêtes analytiques.

2.1. Tables principales

Table	Description	Champs clés
sim_clients	Profils clients simulés uniques	name, age, city, profile
sim_conversations	Conversations complètes avec métadonnées	id, profile, topic, urgency, rag, client_id
sim_turns	Messages individuels (client/advisor)	turn, role, content, latency_ms, intent, modules
sim_sources	Sources RAG utilisées par message	source_id, title, module, score
quality_evaluations	Évaluations de l'Analyste Qualité	global_score, criteria (6), priority, alerts
workforce_decisions	Décisions du Workforce Manager	status, problems, actions, has_escalation

Optimisations appliquées :
• Indexes sur colonnes fréquemment interrogées (profile, topic, created_at, global_score, priority)
• Contraintes de clés étrangères avec CASCADE pour maintenir l'intégrité
• Contraintes UNIQUE pour éviter les doublons (client, conversation+turn+role)
• Type JSONB pour stocker les rapports complets avec possibilité de requêtes JSON

2.2. Vues Business Intelligence

Sept vues matérialisées facilitent l'interrogation des données pour le reporting et l'analyse. Elles pré-calculent les agrégations complexes et les jointures multi-tables, optimisant ainsi les performances des dashboards.

Vue	Objectif	Métriques calculées
v_powerbi_turns	Vue denormalisée pour Power BI	1 ligne par message avec contexte complet
v_bi_full	Vue analytique 360°	Jointure conversation + qualité + workforce
v_daily_kpis	KPIs journaliers agrégés	Score moyen, taux escalade, nb alertes critiques, latence
v_profile_perf	Performance par profil client	Score/critères moyens, taux escalade par profil (8 profils)
v_topic_analysis	Analyse par thématique	Volume, score, risques par topic (12 thématiques)
v_escalation_log	Journal des escalades	Conversations critiques + problèmes + actions déclenchées
v_criteria_scores	Scores détaillés par critère	Format long pour graphiques radar (6 critères qualité)

Exemple de vue v_daily_kpis (extrait SQL) :

```
SELECT DATE(c.created_at) AS day,
    COUNT(DISTINCT c.id) AS nb_conversations,
    ROUND(AVG(q.global_score)::NUMERIC, 1) AS avg_quality_score,
    ROUND(100.0 * COUNT(CASE WHEN w.has_escalation THEN 1 END)
        / NULLIF(COUNT(DISTINCT c.id), 0), 1) AS escalation_rate_pct
FROM sim_conversations c
LEFT JOIN quality_evaluations q ON q.conversation_id = c.id
LEFT JOIN workforce_decisions w ON w.conversation_id = c.id
GROUP BY DATE(c.created_at)
```

3. API Dashboard BI

Le module **backend/routes/dashboard.py** expose 9 endpoints FastAPI interrogeant directement les vues PostgreSQL. Ces endpoints servent les deux dashboards (React et Streamlit) avec des données formatées JSON prêtes à la visualisation.

3.1. Endpoints FastAPI

Endpoint	Méthode	Description
/api/dashboard/kpis	GET	KPIs globaux (nb conversations, score moyen, taux résolution/escalade)
/api/dashboard/criteria	GET	Scores moyens des 6 critères qualité
/api/dashboard/profiles	GET	Performance par profil client (8 profils)
/api/dashboard/trends?days=30	GET	Évolution temporelle des KPIs
/api/dashboard/workforce-stats	GET	Statut et actions Workforce Manager
/api/dashboard/alerts?limit=10	GET	Alertes critiques récentes (vue escalation_log)
/api/dashboard/conversations	GET	Conversations récentes avec filtres (profile, topic)
/api/dashboard/topics	GET	Analyse par thématique (12 topics)
/api/dashboard/client-distribution	GET	Distribution clients par profil avec pourcentages

Mécanisme de fallback : En cas d'absence de PostgreSQL, l'API bascule automatiquement sur une base SQLite locale pour les évaluations qualité, garantissant la continuité du service en mode dégradé.

3.2. Intégration frontend-backend

Le fichier **dt-front/src/api/client.js** encapsule tous les appels API avec gestion d'erreurs centralisée. Un mode *mock* permet le développement frontend sans backend actif (VITE_USE_MOCK=true). Le proxy Vite (/api/* → http://localhost:8000) évite les problèmes CORS en développement.

4. Dashboard Streamlit

Le dashboard Streamlit (**dashboard/app.py**) offre une interface d'analyse avancée connectée directement à PostgreSQL via psycopg2. Il se déploie indépendamment du backend FastAPI et cible les analystes métier et data scientists.

4.1. Architecture et fonctionnalités

• **Connexion directe PostgreSQL** : Utilisation de psycopg2 avec cache Streamlit (@st.cache_resource)
• **Visualisations Plotly** : Graphiques interactifs (line, bar, pie, radar, scatter)
• **Thème sombre personnalisé** : Catppuccin Mocha (cohérence visuelle avec le frontend)
• **Filtrage dynamique** : Par profil, thème, période temporelle
• **Drill-down conversationnel** : Exploration détaillée message par message
• **Rafraîchissement en temps réel** : Bouton de reload avec cache invalidation

4.2. Pages et visualisations

Page	Visualisations	Cas d'usage
■ Vue globale	KPIs cards, histogramme scores, pie chart statuts, bar chart thèmes	Aperçu instantané de la santé du centre
■ Tendances temporelles	Line chart score qualité, bar+line volume/escalades, multi-critères	Analyse de l'évolution dans le temps
■ Analyse par profil	Bar chart horizontal par profil, tableau détaillé avec mini-barres	Identification des profils problématiques
■ Critères qualité	Radar spider chart 6 critères, bar chart avec seuil 0.75, top/bottom 5	Focus sur la qualité des réponses
■■ Alertes & Escalades	Liste expandable conversations critiques avec détails problèmes/actions	Gestion des situations à risque
■ Conversations récentes	Dataframe filtrable + drill-down tour par tour avec métadonnées	Investigation détaillée

Commande de lancement :

```
cd C:/Users/samar/Desktop/PI/dashboard
streamlit run app.py
```

Le dashboard sera accessible sur **http://localhost:8501**

5. Dashboard React intégré

Le composant **dt-front/src/pages/Dashboard.jsx** offre une visualisation intégrée dans l'application web principale. Basé sur Chart.js (react-chartjs-2), il consomme les mêmes endpoints API que le dashboard Streamlit mais s'intègre dans le flux utilisateur du Digital Twin.

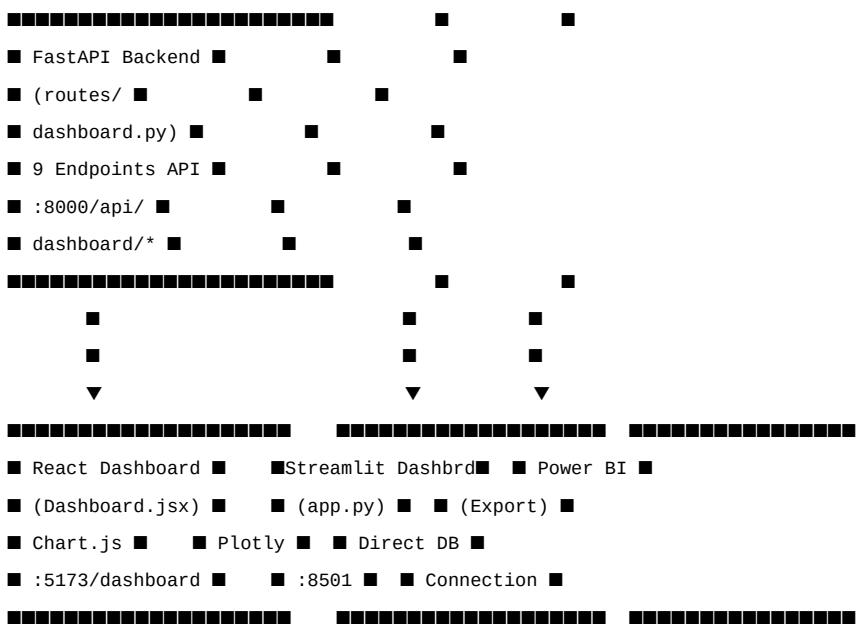
Caractéristiques techniques :
• **Framework** : React 18 + Vite + React Router
• **Graphiques** : Chart.js v4 (Line, Bar, Doughnut, Radar)
• **État** : React Hooks (useState, useCallback, useRef)
• **Styling** : CSS-in-JS avec thème Catppuccin
• **Icônes** : Lucide React (BarChart2, TrendingUp, AlertTriangle...)
• **Responsive** : Grid layout adaptatif avec cartes KPI réutilisables
• **Auto-refresh** : Polling optionnel toutes les 30 secondes

Composants React développés :
• **KpiCard** : Carte KPI avec icône, valeur, tendance et indicateur visuel
• **ChartCard** : Wrapper pour graphiques Chart.js avec titre et hauteur fixe
• **SectionTitle** : En-tête de section stylisé
• **ScoreBadge** : Badge coloré selon seuils (≥75 vert, 60-74 orange, <60 rouge)
• **Dashboard** : Composant principal orchestrant 6 onglets dynamiques

Onglets du dashboard React :
1. **Overview** : KPIs globaux + distribution scores + statut workforce + actions
2. **Performance** : Graphiques par profil client et par thématique
3. **Quality** : Radar des critères + distribution priorités + tendances
4. **Trends** : Évolution temporelle (30 derniers jours)
5. **Alerts** : Liste des conversations escaladées avec drill-down
6. **Details** : Conversations récentes avec filtres et pagination

6. Diagramme d'architecture de la stack BI

Le schéma ci-dessous illustre le flux de données complet depuis les agents jusqu'aux dashboards, en passant par la couche de persistence PostgreSQL et l'API FastAPI.


FastAPI Backend
(routes/
dashboard.py)
9 Endpoints API
:8000/api/
dashboard/*
React Dashboard
(Dashboard.jsx)
Chart.js
:5173/dashboard
Streamlit Dashbrd
(app.py)
Plotly
:8501
Power BI
(Export)
Direct DB
Connection

7. Métriques et KPIs disponibles

Le système expose plus de 40 métriques calculées automatiquement à partir des données de simulation. Elles sont regroupées en 5 catégories principales.

Catégorie	Métriques
Volume & Performance	• Nombre total de conversations • Nombre de clients uniques • Nombre de tours de conversation • Latence moyenne (ms) • Temps de traitement par agent
Qualité	• Score global moyen (/100) • Scores par critère (pertinence, exactitude, groundedness, empathie, clarté, conformité) • Distribution des scores (histogramme) • Taux de conversations ≥75/100
Risques & Alertes	• Nombre d'alertes critiques • Taux de priorité élevée/critique (%) • Taux d'escalade humaine (%) • Conversations nécessitant revue humaine • Classification LoRA (OK/CRITIQUE)
Workforce	• Statut OPTIMAL vs INTERVENTION_REQUISE • Taux de résolution automatique (%) • Actions déclenchées par type (escalade, formation, KB, RAG) • Impact staffing • Modules de formation suggérés
Segmentation	• Performance par profil client (8 profils) • Performance par thématique (12 topics) • Distribution clients par profil (%) • Comparaison min/max/moyenne par segment • Tendances temporelles (jour/semaine/mois)

8. Conclusion et perspectives

Réalisations accomplies :

■ **Base de données robuste** : Schéma PostgreSQL complet avec 6 tables et 7 vues BI optimisées, garantissant la traçabilité end-to-end des conversations et facilitant les analyses complexes.

■ **API Dashboard unifiée** : 9 endpoints FastAPI exposant les données de manière cohérente avec gestion d'erreurs centralisée et mécanisme de fallback SQLite.

■ **Double interface de visualisation** : Dashboard Streamlit pour les analystes métier (6 pages, 15+ graphiques Plotly) et dashboard React intégré pour les utilisateurs finaux (6 onglets, Chart.js).

■ **Intégration complète** : Flux de données automatisé depuis les agents jusqu'aux visualisations sans intervention manuelle.

■ **Performance optimisée** : Vues matérialisées avec agrégations pré-calculées, indexes stratégiques, et cache Streamlit réduisant les temps de requête.

Perspectives d'amélioration :

■ **Export Power BI natif** : Connecteur DirectQuery exploitant v_powerbi_turns pour rapports exécutifs avec drill-through.

■ **Alertes temps réel** : Système de notifications push (WebSocket) pour escalades critiques détectées par le Workforce Manager.

■ **ML Pipeline** : Intégration de modèles prédictifs (régression, classification) pour anticiper les risques d'escalade avant évaluation qualité.

■ **A/B Testing** : Framework pour comparer différentes versions de prompts/modèles avec métriques statistiques (Student t-test, effet de taille).

■ **Audit Trail** : Journalisation des modifications de configuration et des décisions critiques avec conformité RGPD/CNIL.

■ **Multi-tenancy** : Support de plusieurs clients/centres avec isolation des données et dashboards personnalisés par tenant.

Ce système de persistence et de visualisation constitue la **colonne vertébrale analytique** du Digital Twin. Il transforme les simulations brutes en insights actionnables, permettant l'amélioration continue du centre d'assistance client par une approche data-driven rigoureuse.

Samar

Développeur Base de données & BI

November 2026